# A record the record-keeper cannot rewrite.

CreatorProof checks creative work before it is published and seals the result into evidence that survives the party who produced it. This pack is the technical architecture: every layer, every trust boundary, and the reasoning behind each decision.

**SETTLEMENT**

Base, an OP Stack L2 settling to Ethereum L1. Public, permissionless, no privileged writer.

**COMMITMENT**

32 bytes per record via the Ethereum Attestation Service. Nothing else ever crosses the boundary.

**GOVERNANCE**

On-chain member registry, two-step governor transfer, standing regulator-observer seat.

**VERIFICATION**

Any third party can re-derive the whole chain of custody without trusting CreatorProof.

**Blockchain Olympiad Bangladesh 2026**
Finals · Architecture & Governance
Companion to the whitepaper, poster board and pitch deck

**Build**
CreatorProof v0.10.0
Base Sepolia 84532 · EAS · ERC-5192 · EIP-712 · RFC 6962

# 00 How to use this pack



Each sheet carries one diagram with **numbered badges**, the numbered explanation beneath it, and the sentence to say out loud while pointing. The last two sheets are a rapid-fire answer set for the Q&A session, each answer tagged with the diagram to turn to.

## CONTENTS


| | | | | | |
|---|---|---|---|---|---|
| B1 | Why a database cannot do this | Core | B2 | Actors and trust gaps | Context |
| B3 | Anchoring pipeline | Flow | B4 | On-chain / off-chain boundary | Privacy |
| B5 | Contract topology | On-chain | B6 | Transparency log and batching | Scale |
| B7 | Multi-party co-attestation | Trust | B8 | Governance model | Governance |
| B9 | Key management | Security | B10 | Failure and finality | Risk |
| B11 | Independent verification | Proof | B12 | Chain choice and consensus | Platform |
| B13 | Tokenisation of clearance | Assets | B14 | Legacy integration | Adoption |
| B15 | Q&A armory, part one | Defence | B16 | Q&A armory, part two | Defence |


## ARROW GRAMMAR

Line style carries meaning throughout this pack. An arrow is never decoration.

- Data movement
- Decision or control
- Signature binding
- Public commitment

## COLOUR GRAMMAR

PUBLIC PROOF · POLICY OUTCOME · COPY EVIDENCE · ORIGIN EVIDENCE

### OPENING LINE

Our customers have to prove they checked an image *before* they published it. The problem is that we are the ones holding that proof, and we are paid by them.

So the question is not where to store the record. It is how to make a record that we ourselves are unable to change afterwards.

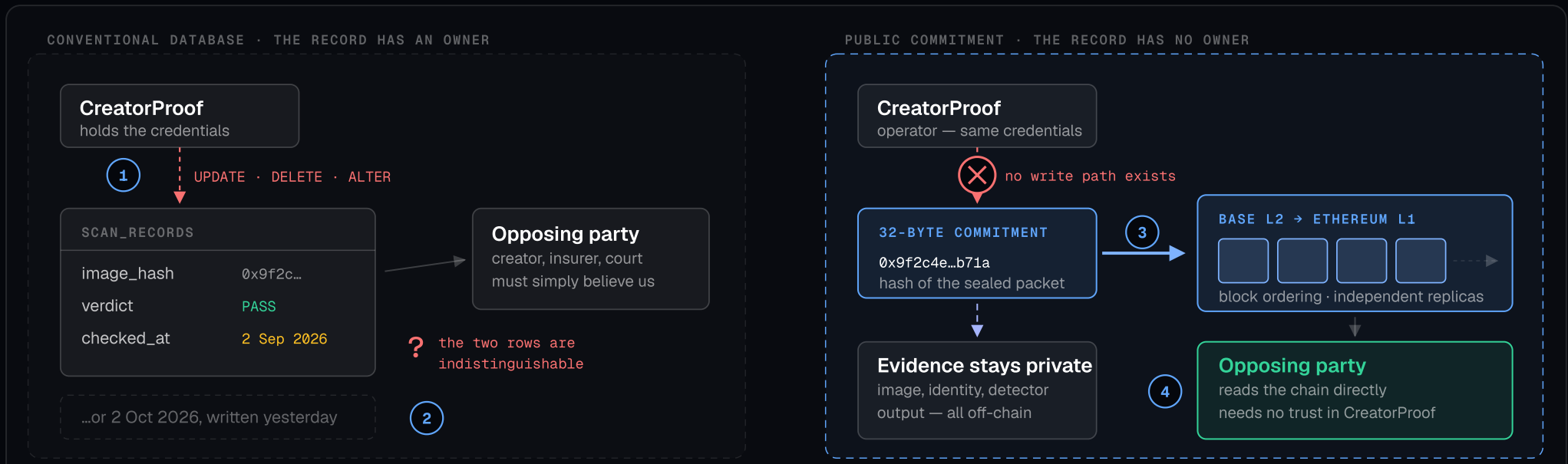
## WHERE EACH BCOLBD CRITERION IS ANSWERED

| EVALUATION CRITERION | PROPOSAL / PROTOTYPE | ANSWERED ON |
|---|---|---|
| Problem & Solution | 30 · 40 | B1 why a database fails · B2 multi-party coordination · B13 value captured · B15–16 the objections in full |
| Architecture | 30 · 20 | B3 pipeline · B4 on-chain vs off-chain · B5 contracts · B6 scale · B12 consensus and platform choice · B14 legacy integration |
| Governance | 30 · 20 | B8 membership, business and infrastructure governance · B7 counterparty commitments · B11 public accountability |
| Privacy & Security Risks | – · 20 | B4 data minimisation and erasure · B9 key management and access control · B10 failure modes |
| Competition & Risks | 20 · – | B1 alternatives compared · B10 technical risk register · B12 platform risk |

# B1 Why a database cannot answer this

THE CORE ARGUMENT

A database answers **“what do you say happened?”** A chain answers **“what can you no longer change about what you said?”** Those are different questions, and only the second one survives a dispute.




B1 · Same operator, same credentials, same evidence. Only the right-hand record is one the operator cannot revise.

1. **The owner is the attacker.** Whoever holds the database credentials can rewrite any row, including the timestamp that the whole defence rests on.
2. **Backdating is undetectable.** A row written on time and a row fabricated a month later are byte-for-byte identical to an outside examiner.
3. **No write path back.** Once the commitment is in a block there is no privileged account, no admin console and no support ticket that can alter it.
4. **Verification without trust.** The opposing party reads the chain themselves. Our cooperation, and our continued existence, stop being required.

**SAY THIS**

We do not need storage. We need an *ordering witness with no privileged writer* — and that is the one thing a database cannot be, by definition.

The moment the record and the interested party share an owner, the record stops being evidence and becomes a claim.

**IF THEY PUSH BACK**

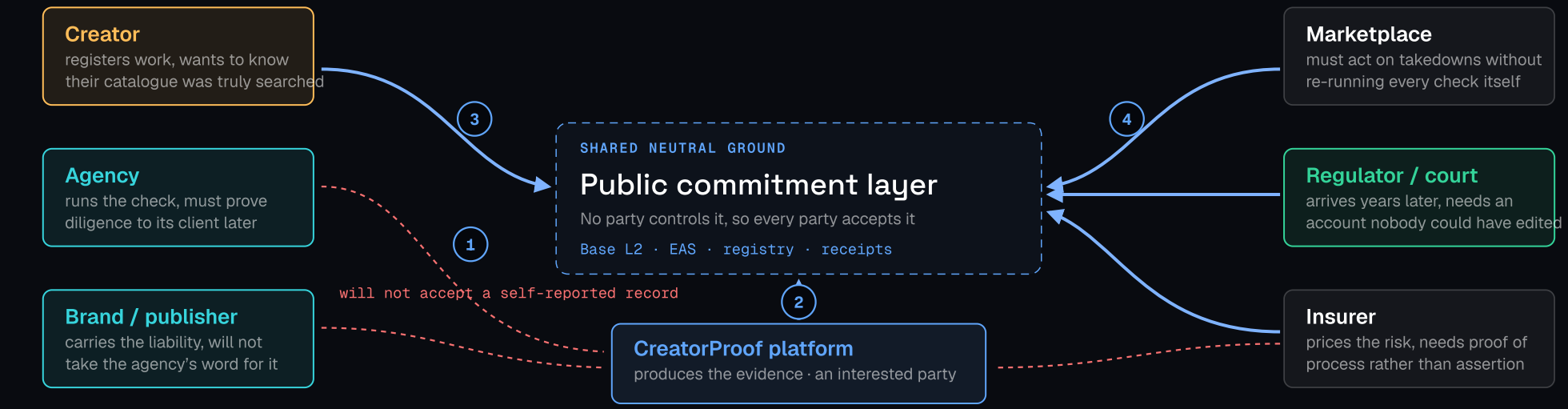
*“Use an append-only table, or WORM storage.”* We hold the credentials that make it append-only, so we can make it not append-only. That relocates the trust; it does not remove it.

*“Use a trusted timestamping authority.”* That is one company: one key to compromise, one entity to subpoena, one business that can fold and take the evidence with it.

*“Let the customer keep their own copy.”* The customer is the interested party. A record that both interested parties can rewrite is worth nothing in a dispute.

# The actors, and who distrusts whom

Publishing a single image touches five parties with genuinely opposed incentives, and no two of them accept each other's records. **Broken lines below are broken trust** — every one of them is a reason this cannot be solved inside any one company's database.




B2 · Red dashed lines are refusals of trust. Blue lines are the one record all six parties will read.

1. **Nobody accepts the interested party's log.** The brand will not trust the agency, and the creator will not trust either of them.
2. **We are not neutral.** CreatorProof is paid by one side, so our own database is disqualified as the shared record for exactly the same reason.
3. **Coordination needs a common reference.** Six parties, six systems, one fact that must mean the same thing to all of them.
4. **The reader arrives late.** Courts and insurers appear years after the fact, when contracts have lapsed and vendors have folded.

SAY THIS

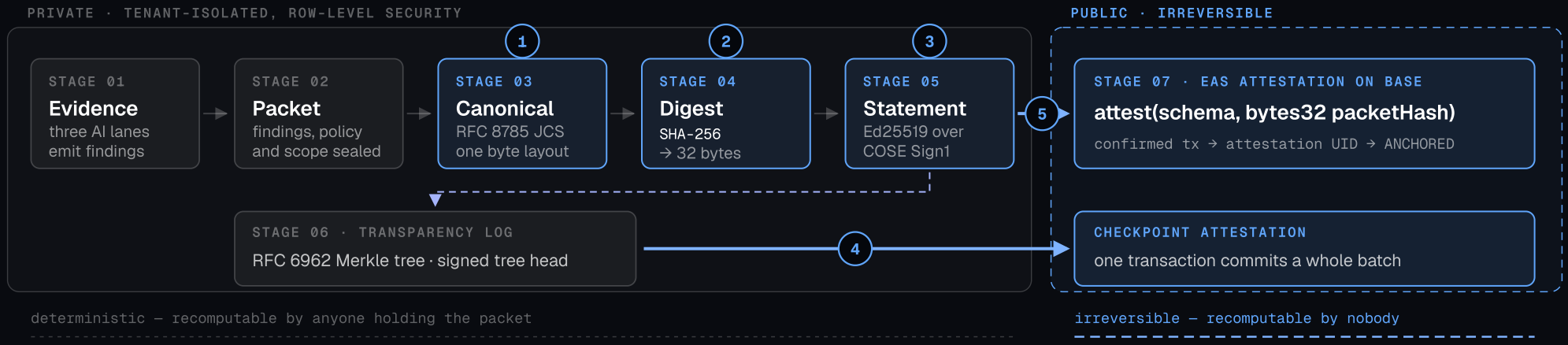
This is a coordination problem between parties whose incentives genuinely conflict, and the referee cannot be one of the players. Every party here needs the same fact to be true, and each of them has a reason to doubt whoever is holding it.

B3

# From uploaded image to public commitment

END-TO-END FLOW

Seven stages turn a raw upload into a fact that cannot be retracted. The critical property is that **every stage is deterministic** — the same evidence always produces the same 32 bytes, so anyone can recompute the commitment and check it themselves.




B3 · Stages 1-6 are reproducible by any holder of the packet. Stage 7 is the only step that cannot be undone.

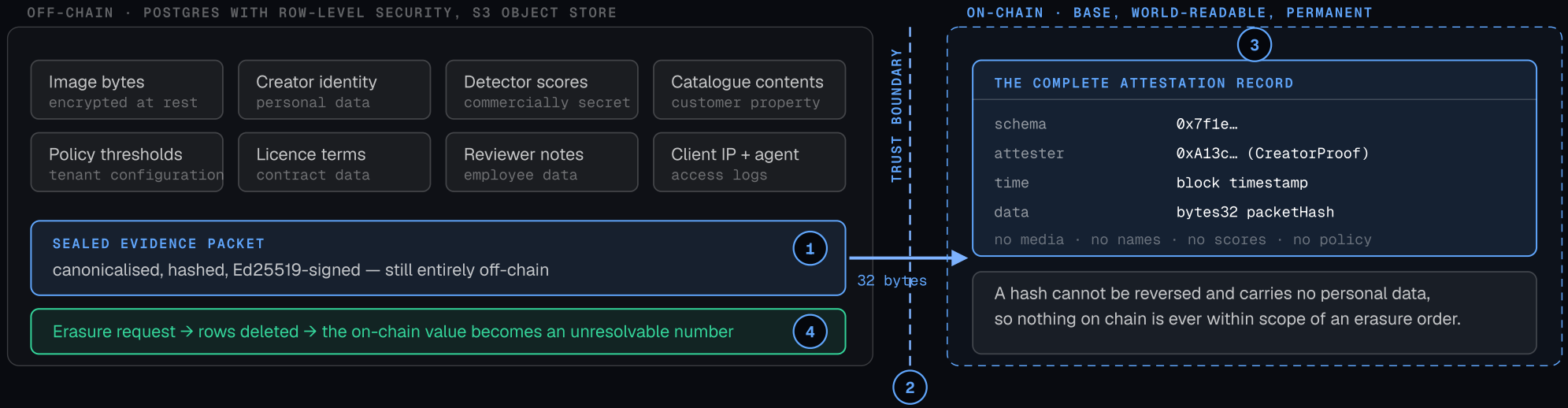
1. **Canonicalisation comes first.** RFC 8785 forces one byte layout per document, so key order or whitespace can never change the hash.
2. **The digest is the whole payload.** Everything the chain will ever see is compressed to 32 bytes here.
3. **Signed before anchored.** Ed25519 proves who asserted it; the chain proves when. Separate keys, separate failures.
4. **The log is the batching point.** Leaves accumulate into a Merkle tree so one transaction can commit thousands of scans.
5. **Anchoring is explicit state.** `anchor_status` is a field on every scan, so a pending anchor is never displayed as a confirmed one.

SAY THIS

Everything left of this line is reproducible — hand anyone the evidence packet and they will derive the identical 32 bytes. Everything right of it is irreversible, and that asymmetry is the entire product.

# The on-chain / off-chain boundary

Exactly one thing crosses this line: **32 bytes with no internal structure**. Everything a regulator would care about stays in tenant-isolated Postgres, which is what makes the right to erasure compatible with a permanent public record.




B4 · Eight categories of sensitive data on the left, one opaque number on the right.

1. **Sealed, then hashed.** The packet is canonicalised and signed off-chain; only its digest is ever eligible to cross.
2. **One direction, one payload.** There is no read path from chain back to data, and no second field to leak through.
3. **The full on-chain record.** Everything a block explorer can show is listed here — and none of it identifies anyone.
4. **Erasure still works.** Deleting the off-chain rows destroys the only pre-image, leaving an orphaned number.

**SAY THIS**

Nothing personal ever reaches the chain. Erasure works *because* of that split — delete the off-chain record and the on-chain hash becomes a meaningless 32 bytes that can never be reversed into the data.

We satisfy the right to erasure and keep the timestamp intact at the same time.

**IF THEY PUSH BACK**

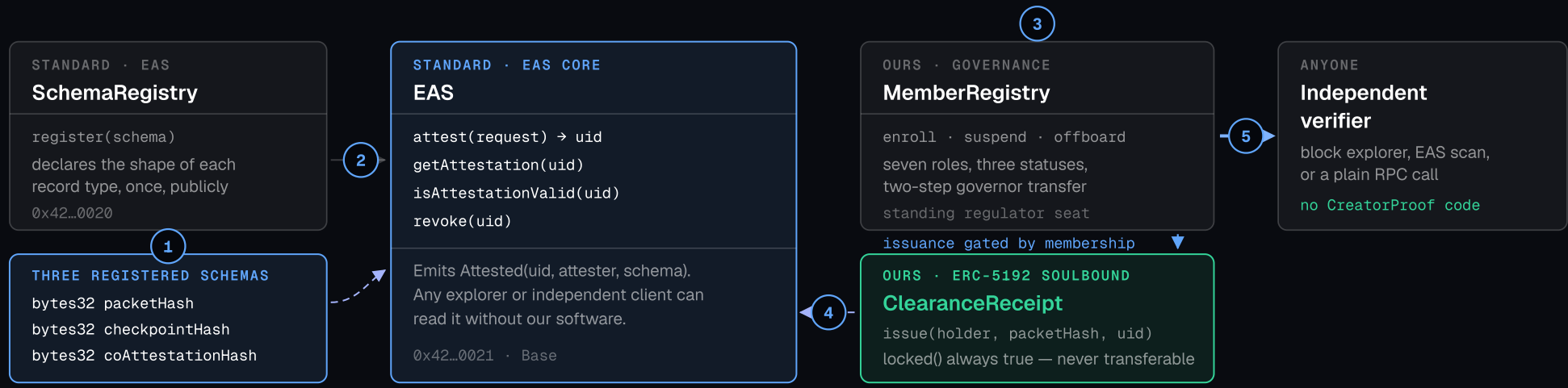
*"A hash of personal data is still personal data."* Only where the pre-image space is small enough to brute-force. Ours is a multi-kilobyte JSON document containing high-entropy fields, so it is not enumerable.

*"Which regulation are you claiming compliance with?"* GDPR Article 17 and the Bangladesh Personal Data Protection framework both attach to identifiability; an unresolvable digest is out of scope once its pre-image is destroyed.

# B5 Contract topology



Four contracts, none of them bespoke cryptography. We build on the **Ethereum Attestation Service** so that a third party can verify our records with tooling they already trust, instead of an audit of code we wrote ourselves.




B5 · Two standard EAS contracts we did not write, two small governance contracts we did.

1. **Three record types, declared publicly.** Evidence packets, log checkpoints and counterparty commitments each get their own schema.
2. **We wrote no cryptography.** EAS is deployed, audited and widely used; our contracts only handle membership.
3. **Governance gates issuance.** A receipt can only be minted for an address the registry currently lists as active.
4. **Receipts point at attestations.** Every receipt stores the packet hash and the attestation UID, so the two layers cannot drift apart.
5. **Verification needs nothing of ours.** A judge can check any record on a public explorer while we are offline.

SAY THIS

We deliberately wrote as little on-chain code as possible. Using the Ethereum Attestation Service means a counterparty verifies us with tooling they already trust, instead of auditing a contract written by the company they are checking up on.

# Transparency log, and why cost stops mattering

Anchoring every scan individually would not scale. Instead each record becomes a leaf in an **RFC 6962 Merkle tree**, and one transaction commits the entire batch — while every individual record keeps its own independently checkable proof.




B6 · The blue path is everything needed to prove one record was in the tree the chain committed to.

1. **Domain-separated hashing.** Leaves and internal nodes use different prefixes, which blocks second-preimage attacks on the tree.
2. **One root represents everything.** Change any single record and the root changes, so the published root pins the whole batch.
3. **One transaction, whole batch.** Gas is paid once per checkpoint rather than once per customer scan.
4. **Cost stops being an argument.** At this marginal price we anchor every record, not only the disputed ones.

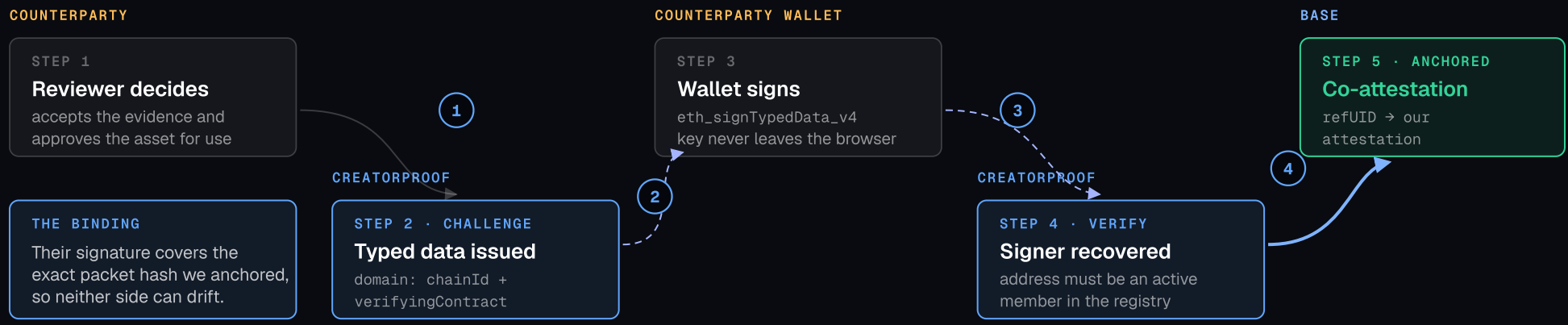
**SAY THIS**

A record you only create once you are already in trouble is worthless, so the anchoring has to be cheap enough to run on every single check. Batching is what takes the cost per record to effectively zero while each record keeps its own proof.

# B7 Multi-party co-attestation

NOT JUST OUR WORD

A record we sign alone is still a record by an interested party. So the counterparty who *acted* on the result signs it too, from their own wallet, using **EIP-712 typed data** — and their commitment is bound on-chain to ours.




B7 · The counterparty's private key never leaves their browser; only the recovered address is stored.

1. **The decision triggers it.** Co-attestation is requested at the moment a human accepts the result, not retroactively.
2. **Replay is impossible.** The typed-data domain pins chain ID and the registry address, so a signature cannot be reused elsewhere.
3. **Membership is enforced.** We recover the signing address and reject it unless the registry lists it as active.
4. **Bound, not merely adjacent.** `refUID` makes the counterparty commitment a child of ours, permanently linked on chain.

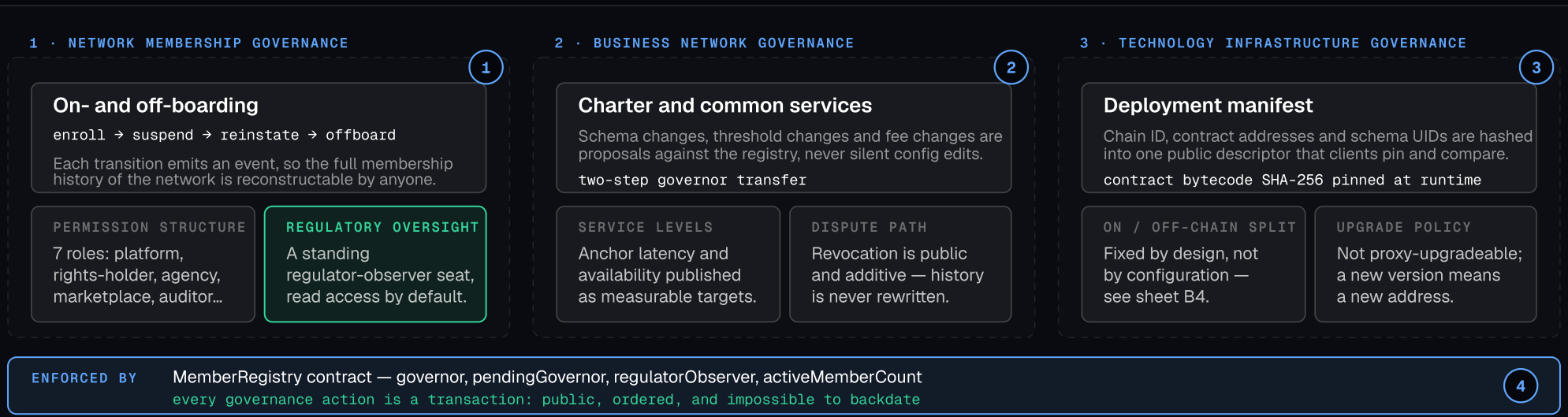
SAY THIS

This is what stops the record being self-serving. The brand that relied on the result signs the same 32 bytes we did, from their own key, and that pair of signatures is what a court would actually find persuasive.

# B8 Governance across all three layers

NETWORK · BUSINESS · INFRASTRUCTURE

Governance is where most consortium designs quietly become “whatever the founding company decides”. Ours is enforced by contract, not by policy document — **membership, roles and removal are all on-chain and publicly auditable**.




B8 · The three governance domains the BCOLBD checklist names, each mapped to an on-chain mechanism.

1. **Removal is as public as joining.** Suspension and off-boarding are transactions, so a member cannot be quietly deleted from history.
2. **No single-transaction takeover.** Governor transfer is propose-then-accept, so a stolen key cannot instantly capture the network.
3. **Clients verify the deployment.** If we swapped a contract address, every pinned client would notice immediately.
4. **Policy that cannot be ignored.** These are not commitments in a PDF; they are the only code paths that exist.

SAY THIS

The honest failure mode of most consortium networks is that governance lives in a document while control lives in one company’s admin panel. Here the registry contract *is* the governance, so our own power over the network is bounded and visible.

# B9 Key management and access control



Three key families with different jobs, different lifetimes and different blast radii. The design rule is that **no single key compromise can rewrite history** — the worst case is that new records stop being trustworthy, never that old ones change.

FAMILY 1 · EVM ATTESTER KEY ①

**secp256k1 · writes to Base**

Held in a cloud KMS. The private key is never present in application memory.

`lease mutex stops two workers signing`

BLAST RADIUS IF STOLEN

Can publish false new records. Existing ones stay untouched.

FAMILY 2 · STATEMENT SIGNING KEY ②

**Ed25519 · signs evidence**

Separate from the EVM key, so chain access and evidence authorship differ.

`rotated on schedule · key history public`

BLAST RADIUS IF STOLEN

Can forge statements, but cannot backdate them past the chain.

FAMILY 3 · COUNTERPARTY WALLET KEYS ③

**held by the customer**

We never hold, escrow or recover a customer key. We only store addresses.

`signing happens in their browser`

BLAST RADIUS IF STOLEN

Confined to one member; the registry can suspend the address.

ACCESS CONTROL, OFF-CHAIN ④

Per-tenant API keys · PostgreSQL row-level security with a NOBYPASSRLS runtime role · reviewer actions attributed and logged

B9 · Separated key families mean a single compromise degrades future trust without corrupting the past.

① **The chain key never touches our code.** Signing happens inside the KMS, and a database lease stops two workers racing the same nonce.

② **Rotation is expected, not exceptional.** Old public keys stay published so historical statements remain verifiable after a rotation.

③ **We are not a custodian.** Holding no customer keys removes the single most attractive target in the system.

④ **Isolation is enforced by the database.** Row-level security means a bug in application code still cannot cross a tenant boundary.

SAY THIS

We assumed a key would eventually be compromised and designed for what happens next. Because the past is already committed on a public chain, the worst an attacker can do is pollute the future — and that becomes visible immediately.

# B10 Failure, finality and honest degradation

TECHNICAL RISK

Chains stall, RPC providers drop and gas spikes. The system therefore treats anchoring as a **durable job with visible state**, and it will report a pending anchor as pending rather than quietly claiming success.




1. **Idempotent by construction.** Re-submitting the same packet cannot produce a second attestation for the same record.
2. **Failure is a state, not an exception.** Gas spikes and RPC outages route into a bounded retry loop with provider failover.
3. **Service continues while the chain is unreachable.** Leaves keep accumulating locally and are anchored once connectivity returns.
4. **We never overstate a proof.** A pending anchor is displayed as pending, which is the only version of this that survives cross-examination.

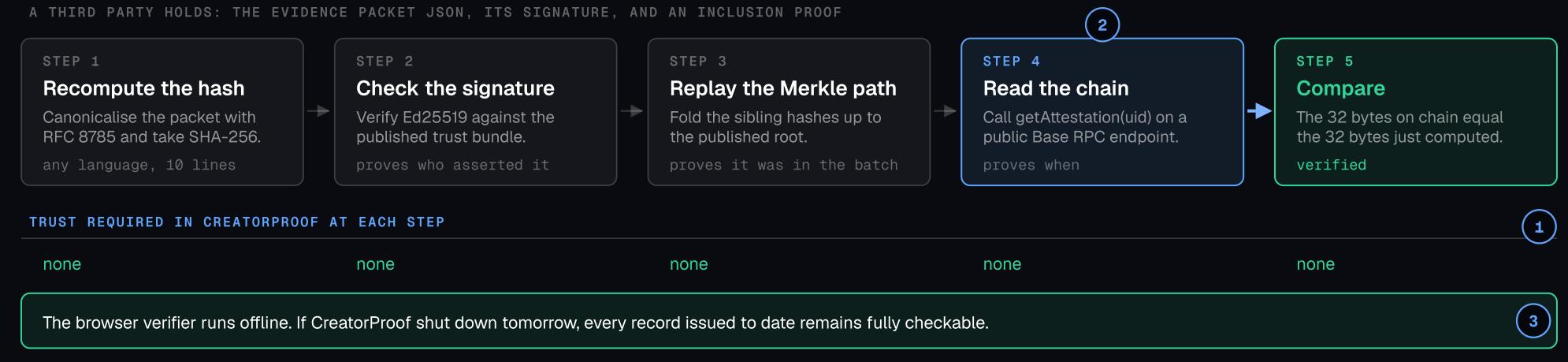
SAY THIS

The dangerous failure here is not downtime, it is a system that reports a proof it does not have. So anchor state is a first-class field on every result, and the product would rather show you “pending” than a comfortable lie.

# B11 Verifying a record with zero trust in us



This is the test the whole architecture exists to pass. A stranger holding only an evidence packet can confirm the entire chain of custody **without our servers, our software, or our cooperation**.




1. **Zero trust at every step.** Each stage consumes only public data or data the verifier already holds — our cooperation is never an input.
2. **Any RPC endpoint works.** The verifier picks their own node provider, so we cannot serve them a doctored view of the chain.
3. **It survives us.** Business continuity is not a promise here; the evidence simply does not depend on the company continuing to exist.

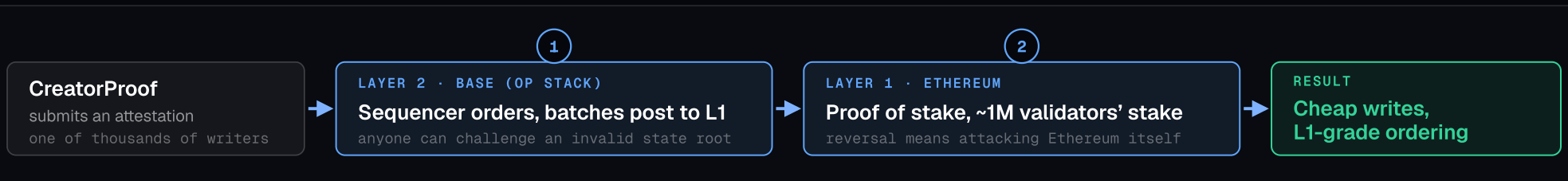
SAY THIS

Ask the hardest version of the question — what if CreatorProof is bankrupt, or lying, or gone? Every record we have ever issued still verifies, and that is the difference between a service and an actual proof.

# B12 Why Base, and how consensus works here

PLATFORM CHOICE

We need public settlement, negligible cost and a standard tooling ecosystem. **Base is an OP Stack layer 2 that settles to Ethereum**, so transactions are cheap while final ordering inherits Ethereum's proof-of-stake security.




**WHY NOT A PERMISSIONED CHAIN** 3

A consortium chain is governed by its members — and in year one that consortium is effectively us.
Permissionless settlement is the only version where "we could not have faked this" is a fact rather than a promise.

B12 · Cost lives on layer 2; the security guarantee lives on layer 1.

| OPTION | WHO CAN REWRITE IT | COST PER RECORD | WHY IT WAS REJECTED |
|---|---|---|---|
| PostgreSQL | We can | ~0 | The record and the interested party share an owner, so it proves nothing in a dispute |
| Append-only / WORM | We can, with effort | ~0 | We still administer the mechanism that makes it append-only |
| RFC 3161 timestamp authority | One company can | ~0.01 USD | Single point of trust, single subpoena target, single business-continuity risk |
| Permissioned consortium chain | The consortium can | infra cost | Requires the counterparties to already exist and cooperate — a chicken-and-egg failure |
| **Base L2 + EAS** | Nobody | ≈0.000002 USD | Chosen — public settlement, standard tooling, cost low enough to anchor every record |

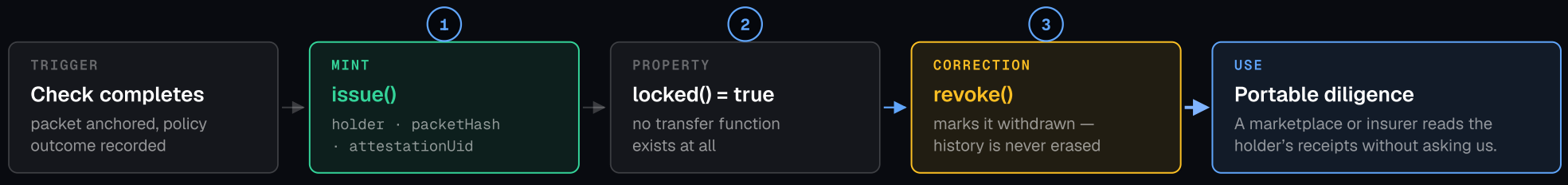
**SAY THIS**

Every alternative we considered fails the same test: somebody retains the ability to change the record. Base is where that answer becomes "nobody", at a price that lets us do it on every single check rather than only the contested ones.

# B13 Tokenising clearance, not speculation



The tokenised asset here is a **clearance receipt**: proof that a specific party ran a specific check. It is deliberately **soulbound under ERC-5192**, because a diligence record that can be sold to someone else is not a diligence record.




**WHY SOULBOUND**

A transferable receipt would create a market in other people's diligence. Binding it to the checking party is what preserves its evidentiary meaning.

**WHAT WE DELIBERATELY DID NOT BUILD** 4

No fungible token, no fundraise instrument, no staking and no speculative secondary market. The asset carries information value, and value does not have to mean money.

B13 · A non-financial token whose only job is to make one party's diligence portable and checkable.

1. **Bound to the evidence.** Each receipt stores the packet hash and attestation UID, so it cannot be pointed at a different result later.
2. **Non-transferable by construction.** ERC-5192 with no transfer path means it cannot be traded, lent or laundered.
3. **Withdrawal is additive.** Revocation records that a receipt was withdrawn and when; nothing is deleted.
4. **No token economics.** Judges are right to be sceptical of gratuitous tokens — this one carries information, not price.

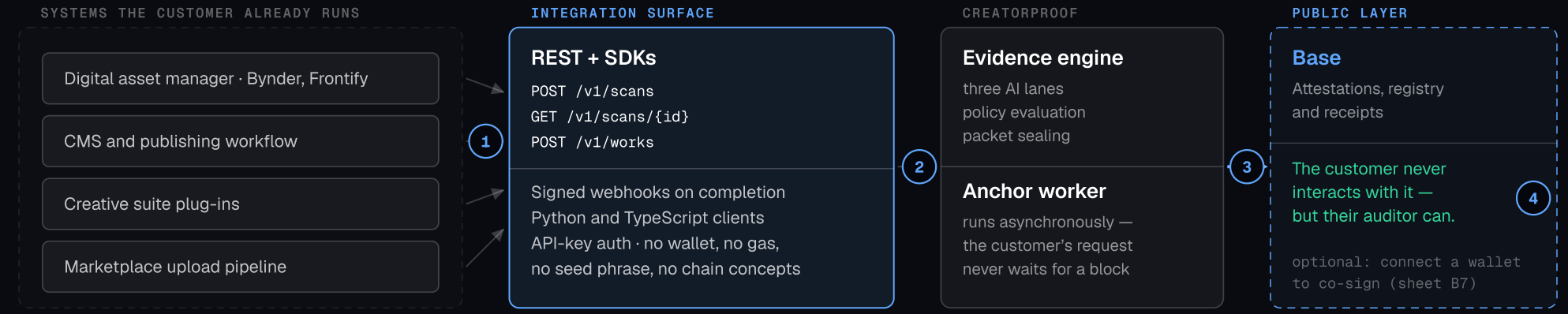
**SAY THIS**

We tokenised the one thing that genuinely benefits from being portable: the fact that a named party checked something on a given date. Making it soulbound is the whole point, because diligence you can buy from someone else is not diligence.

# B14 Fitting into systems that already exist

LEGACY INTEGRATION

No customer will rebuild their content pipeline for us, and none of them want to touch a wallet. So the blockchain is **invisible at the integration surface**: a normal REST call in, a normal webhook out, and a verifiable receipt underneath.




B14 · Adoption cost is one HTTP call. The chain is an implementation detail until someone needs to audit.

1. **Ordinary API, ordinary auth.** Teams integrate this the way they integrate any other vendor, in an afternoon.
2. **Anchoring is off the request path.** The scan returns immediately; the chain write completes behind it.
3. **No crypto literacy required.** No wallets, no gas, no seed phrases anywhere in the customer's workflow.
4. **Depth is opt-in.** Customers who want to co-sign can connect a wallet; everyone else gets the benefit without the concept.

SAY THIS

The single biggest reason blockchain projects fail commercially is that they ask the customer to change how they work. Ours asks for one HTTP request, and the customer can go their whole relationship with us without ever learning what Base is.

# B15 Q&A armory — the hard questions



Each answer is two or three sentences, followed by the sheet to turn to. Answer, then point — do not read the sheet aloud.

QUESTION 01

### Why blockchain? Why not just PostgreSQL?

A database answers “what do you say happened?”; a chain answers “what can you no longer change about what you said?”. Our customer’s defence is a date, and we are the ones holding it while being paid by them. We need an ordering witness with no privileged writer, which is the one thing a database cannot be.

Turn to **B1** · left panel first, then the right

QUESTION 02

### An append-only table would do the same job.

We hold the credentials that make it append-only, so we can also make it not append-only. That relocates the trust into our operations team rather than removing it, and an outside examiner cannot audit our operations team.

Turn to **B12** · comparison table, row two

QUESTION 03

### You are only storing a hash. That is trivial.

Correct, and the minimalism is the design. Because it is 32 bytes it costs a fraction of a cent, so we anchor every check rather than only disputed ones — and a record created after trouble starts is worthless. The chain also carries the membership registry and the co-attestations.

Turn to **B6** · then **B5** for the rest of the on-chain surface

QUESTION 04

### What about GDPR? You cannot delete from a chain.

Nothing personal ever reaches the chain — only a 32-byte hash. Erasure works because of that split: delete the off-chain rows and the on-chain value becomes an unresolvable number. We satisfy erasure and keep the timestamp at the same time.

Turn to **B4** · point at the vertical boundary

QUESTION 05

### Is this actually decentralised, or just you writing to a chain?

Two mechanisms make it more than that. Membership, roles and removal live in a contract with a two-step governor transfer and a standing regulator seat, and the counterparty who acted on a result co-signs it from their own wallet.

Turn to **B8** · then **B7**

QUESTION 06

### Why Base rather than Ethereum directly, or a private chain?

Base is an OP Stack layer 2 that settles to Ethereum, so we get L1 ordering security at L2 prices. A private chain is governed by its members, and in year one those members are effectively us — which reintroduces exactly the problem we are solving.

Turn to **B12** · the stack, left to right

QUESTION 07

### What is the consensus mechanism you depend on?

Base sequences transactions and posts batches to Ethereum, where proof-of-stake validators provide final ordering and fault proofs let anyone challenge an invalid state root. Reversing one of our records means attacking Ethereum itself.

Turn to **B12** · layer 1 box

QUESTION 08

### What happens if the chain is down or gas spikes?

Anchoring is a durable job with persisted state, exponential backoff and RPC failover across providers. The local Merkle log keeps accepting records meanwhile, and every result exposes its anchor status rather than claiming a proof it does not yet have.

Turn to **B10** · the retry loop

# B16 Q&A armory — business, risk and scope



Commercial and adversarial questions. Where you do not know something, name the boundary precisely and move on.

QUESTION 09

### Who pays for the gas, and does it scale?

We do, and it is negligible because one checkpoint transaction commits an entire batch of records. The marginal cost per record is around two millionths of a dollar, so anchoring cost never becomes a reason to stop anchoring.

Turn to **B6** · the economics panel

QUESTION 10

### What stops you anchoring a false record in the first place?

Nothing on the chain — it proves timing and integrity, not truthfulness. What constrains it is the counterparty co-signature, an evidence packet anyone can re-derive, and the fact that a false record is permanent once written.

Turn to **B7** · then **B11**

QUESTION 11

### Why does this need a token at all?

It does not need a financial one, and we did not build one. The only token is a soulbound ERC-5192 receipt proving a named party ran a specific check, which cannot be transferred precisely because diligence you can buy from someone else is not diligence.

Turn to **B13** · the right-hand panel

QUESTION 12

### Your customers do not want to touch crypto.

They never do. Integration is one REST call with an API key — no wallet, no gas, no seed phrase — and anchoring happens asynchronously behind the response. The chain only becomes visible when somebody needs to audit.

Turn to **B14** · the integration surface column

QUESTION 13

### What is your single biggest technical risk?

Dependence on one layer-2 ecosystem. Because we write only standard EAS attestations and publish a hashed deployment manifest, migrating to another EVM chain re-points addresses without changing the evidence format or invalidating past records.

Turn to **B8** · infrastructure governance column

QUESTION 14

### What if someone steals your signing key?

They could publish false new records, but they could not alter a single existing one, because the past is already committed. Key families are separated by job, the chain key lives in a KMS, and we hold no customer keys at all.

Turn to **B9** · the blast-radius lines

QUESTION 15

### Who are your competitors, and why do they lose?

Content-ID style matchers and AI-detector APIs both return a score and keep the record inside their own database. We compete on the artifact rather than the score: the evidence survives us, and theirs does not survive them.

Turn to **B11** · the “none” row

QUESTION 16

### Where does this genuinely not work?

We prove a check was run, its inputs and its timing. We do not prove ownership and we do not issue legal findings of infringement — those stay human and judicial. Naming that boundary is what makes the rest credible.

Turn to **B1** · restate what the record actually asserts

CLOSING LINE FOR THE SESSION

Every question here returns to one point: we built the record so that our own word is never load-bearing. If you do not trust us, the evidence still stands.